

Experimental Report

Submodular Optimization and Greedy Approximation for Distributed Influence Maximization

Yixin Chen Bichi Zhang Yaoqin Ye
CS240: Algorithm Design and Analysis, Spring 2026

Abstract. This project studies influence maximization: choosing a limited seed set whose diffusion reaches as many network nodes as possible. We connect the application to monotone submodular maximization, the $(1 - 1/e)$ greedy guarantee, CELF lazy evaluation, and a communication-constrained GreeDI-style method. Under Independent Cascade (IC) simulation, CELF preserves essentially the same spread as Monte Carlo greedy while reducing average large-preset spread evaluations from 1026.5 to 266.2 and runtime from 4.67 s to 0.77 s. Distributed candidate sharing exposes a quality-runtime-communication tradeoff, while weighted degree is unusually competitive under the weighted-cascade rule.

1 Background and Problem Formulation

Influence maximization appears in viral marketing, public-health intervention, and misinformation analysis. Kempe, Kleinberg, and Tardos [1] proved NP-hardness under the IC and Linear Threshold models and showed that expected spread is monotone and submodular. Nemhauser, Wolsey, and Fisher [2] therefore imply a $(1 - 1/e)$ approximation for exact greedy maximization under a cardinality constraint. CELF [3] accelerates greedy through lazy marginal-gain evaluation, while GreeDI [4] limits global candidate exchange. Later scalable approaches include TIM/IMM [5, 6], PaC-IM [7], distributed submodular algorithms [8], and GreediRIS [9].

Let $G = (V, E, p)$ be directed, with activation probability p_{uv} on edge (u, v) . Given budget k , the objective is

$$\max_{S \subseteq V, |S| \leq k} \sigma(S),$$

where $\sigma(S)$ is expected spread after IC diffusion. Each newly active node gets one independent attempt to activate every inactive out-neighbor. For $A \subseteq B$ and $v \notin B$,

$$\sigma(A \cup \{v\}) - \sigma(A) \geq \sigma(B \cup \{v\}) - \sigma(B),$$

so marginal returns diminish. Exact spread is expensive, and our implementation estimates it with Monte Carlo simulation.

2 Representative Methods

Centralized greedy. Starting from $S_0 = \emptyset$, round t selects

$$v^* = \arg \max_{v \in V \setminus S_t} [\hat{\sigma}(S_t \cup \{v\}) - \hat{\sigma}(S_t)], \quad S_{t+1} = S_t \cup \{v^*\}.$$

With n nodes, m edges, and R simulations per estimate, one IC rollout costs $O(n+m)$ and naive greedy costs approximately $O(knR(n+m))$.

CELF lazy greedy. CELF stores cached marginal gains in a max-priority queue. Since a stale gain upper-bounds its current value, only the queue leader is recomputed until the top entry is current for the present seed-set size. It targets the same greedy sequence with far fewer spread evaluations.

GreeDI-style candidate sharing. Nodes are randomly partitioned among m workers. Worker i runs local greedy and returns at most q candidates; the coordinator forms $C = \bigcup_{i=1}^m C_i$ and runs greedy over C . We test $m \in \{2, 4, 8\}$ and $q \in \{k, 2k, 5k\}$. Communication is measured by the number of transmitted candidates.

3 Implementation and Setup

The code implements IC simulation; random, weighted-degree, and PageRank baselines; centralized greedy and CELF; random partitioning and GreeDI-style merging; graph construction; and CSV/figure generation. The quick preset uses Erdos–Renyi (ER), Barabasi–Albert (BA), and Watts–Strogatz (WS) graphs with $n \in \{40, 70\}$ plus the 34-node Karate Club graph. The large preset uses the same synthetic families with $n \in \{50, 100, 200\}$ plus Karate; $k = 10$ for synthetic graphs and $k = 5$ for Karate. Each final seed set is re-evaluated with a common Monte Carlo seed for the same graph case. Figure 1 shows the quick-preset graph structures.

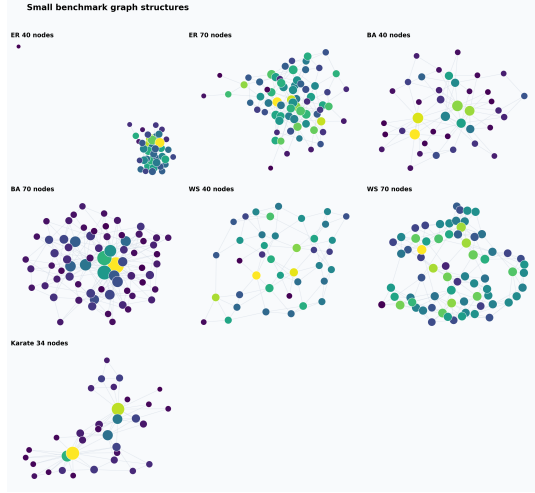


Figure 1: Quick-preset graph structures. Node color and size encode weighted out-degree.

4 Experimental Evaluation

The large preset better reflects the final project state than the quick smoke test. Table 1 summarizes the core methods across ten graph cases. Ratios are computed relative to centralized greedy on the same graph after common-seed re-evaluation.

Table 1: Average core-method performance across all large-preset graph cases.

Algorithm	Spread	Ratio	Runtime (s)	Evaluations
Random	31.59	0.79	0.00	1.0
Weighted degree	43.71	1.07	0.01	1.0
PageRank	35.19	0.89	0.00	1.0
Greedy	40.43	1.00	4.67	1026.5
CELF	40.05	0.99	0.77	266.2

Weighted degree exceeds greedy on average because weighted cascade gives structurally central nodes high influence potential. This does not contradict the approximation theorem: the theorem compares exact greedy with the exact objective, whereas our implementation selects with noisy Monte Carlo estimates and re-evaluates final seed sets with another common seed. CELF is the clearest theory-to-practice result: Table 1 shows ratio 0.99, 74.1% fewer spread evaluations, and an approximately $6.1\times$ runtime reduction relative to naive greedy.

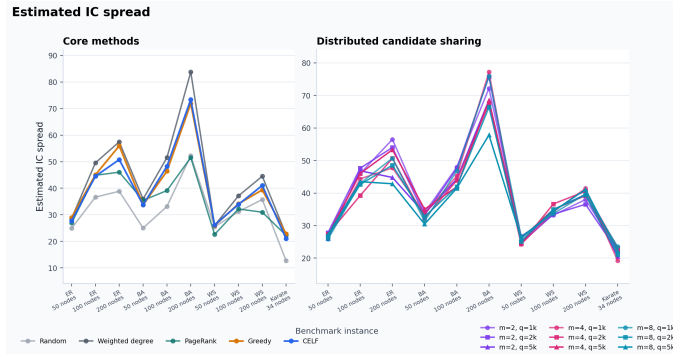


Figure 2: Estimated IC spread on the large preset.

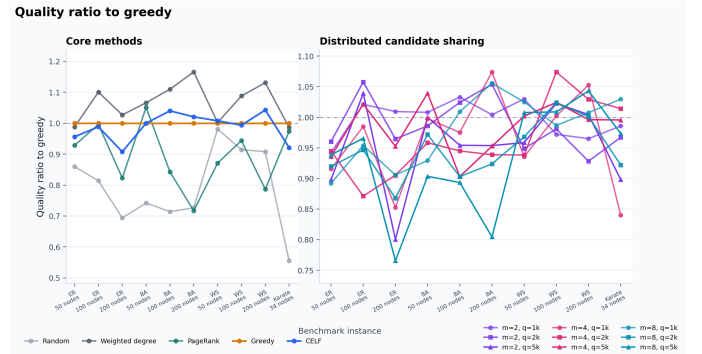


Figure 3: Quality ratio relative to centralized greedy.

Figures 2 and 3 show that CELF closely follows greedy across instances, whereas distributed settings vary more. Structural baselines are especially competitive on BA graphs, whose hubs align with weighted-cascade influence. The spread chart also shows that all methods respond similarly to graph size and family, while the ratio chart isolates instance-level deviations from greedy.

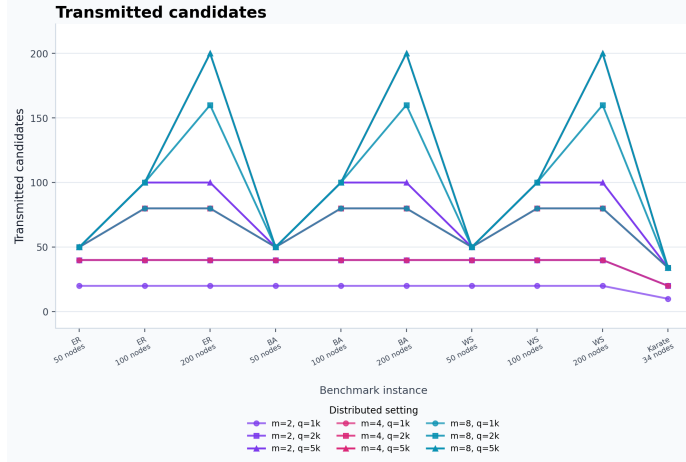
4.1 Distributed Communication Tradeoff

Table 2 reports the distributed variants. The notation $(m, q/k)$ denotes the number of partitions and the local candidate budget relative to k .

Table 2: Average distributed performance under different partition and communication budgets.

Setting	Spread	Ratio	Runtime (s)	Evaluations	Candidates
(2, 1)	40.43	1.00	1.54	1130.5	19.0
(2, 2)	40.28	0.99	3.78	2114.5	38.0
(2, 5)	38.40	0.95	9.89	3984.1	78.4
(4, 1)	39.39	0.96	1.54	1234.5	38.0
(4, 2)	38.56	0.96	3.43	2072.1	66.4
(4, 5)	39.48	0.98	6.44	3068.1	108.4
(8, 1)	39.70	0.98	1.66	1362.1	66.4
(8, 2)	37.95	0.94	2.86	1924.9	96.4
(8, 5)	36.72	0.93	3.12	2080.9	108.4

As Table 2 shows, the smallest pool, $(m, q/k) = (2, 1)$, ties centralized greedy on average with only 19 transmitted candidates. Increasing q raises communication and usually total work, but quality is not monotone because both local and coordinator selection depend on noisy spread estimates. Figure 4 visualizes this communication cost directly.

**Figure 4:** Communication proxy: number of transmitted candidates on the large preset.

Consistent with Table 2, the $q = 5k$ settings transmit the largest pools, yet their average quality ratios remain between 0.93 and 0.98. Communication therefore grows predictably with the budget, whereas solution quality remains sensitive to estimation noise.

4.2 Runtime and Evaluation Counts

Centralized greedy averages 1026.5 spread evaluations on the large preset. CELF reduces this to 266.2 by reusing marginal-gain upper bounds. Distributed variants often perform more total evaluations because every worker runs a local greedy phase and the coordinator runs another phase. These totals are attractive only when workers run in parallel or when full-graph search is impossible because of memory, ownership, privacy, or communication constraints. The present implementation models the distributed structure but does not execute workers in parallel.

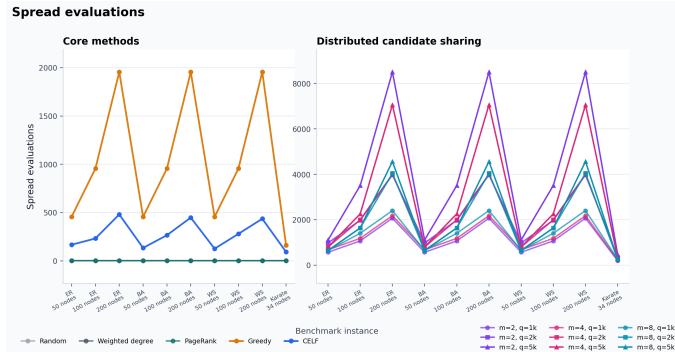
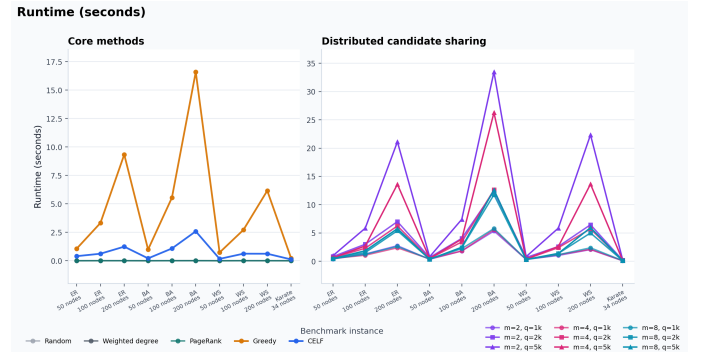
**Figure 5:** Number of spread evaluations. CELF sharply reduces naive greedy work.**Figure 6:** Wall-clock runtime. Distributed totals include worker and coordinator phases.

Figure 5 confirms CELF’s reduction in repeated spread estimation, while Figure 6 shows the corresponding wall-clock pattern. Core-method cost rises sharply with graph size. Distributed totals also include repeated local and coordinator searches, so they represent aggregate work rather than parallel elapsed time.

4.3 Main Findings

- Weighted degree is a strong baseline under the weighted-cascade probability rule.
- CELF preserves almost all greedy spread while reducing evaluations by 74.1% and runtime by about $6.1\times$.
- The best average distributed setting, $(2, 1)$, matches greedy with a small communication budget.
- More transmitted candidates do not guarantee better spread when local and coordinator estimates are noisy.

5 Deviations, Limitations, and Extensions

The proposal considered synthetic graphs with $n \in \{500, 1000, 5000\}$ and real networks such as Wiki-Vote or NetHEPT. The final reproduction stops at 200 synthetic nodes plus Karate so that Monte Carlo greedy runs reliably in the course environment. This keeps the algorithmic mechanisms transparent but does not support strong scalability claims. The experiment runner accepts custom synthetic sizes through `--sizes`; the submitted CSV files use the documented quick and large presets.

Quality versus structure. Approximation guarantees alone do not determine empirical rankings. The graph family and activation-probability rule can make a simple structural heuristic unusually strong. Greedy remains the conceptual reference point, but finite-sample spread estimation can alter its selected sequence.

Communication versus estimation. Increasing q exposes the coordinator to more candidates and raises selection cost. In principle, the larger union can improve quality; in practice, it also creates more opportunities for Monte Carlo estimation error. The non-monotone trend in Table 2 therefore reflects both communication budget and estimator stability.

Reproducibility. `simulation.py` implements IC and spread estimation; `baselines.py` contains the three baselines; `greedy.py` contains greedy and CELF; `distributed.py` handles partitions and sharing; `graphs.py` builds benchmarks; and `experiments.py` writes figures and CSV files. All final seed sets for a graph case are re-evaluated with a common random seed.

Extensions. Useful next steps include deterministic partitioning, larger real datasets, reverse-reachable-set maximum coverage, additional Monte Carlo trials, confidence intervals, and true parallel worker execution. These changes would separate algorithmic work from elapsed-time speedup more clearly.

6 Conclusion

Influence maximization provides a compact bridge from modern network applications to classical algorithm design. IC spread is monotone and submodular; greedy supplies an approximation guarantee; CELF converts diminishing returns into a practical acceleration; and Greedy-style candidate sharing exposes communication constraints. At course scale, performance depends not only on the approximation algorithm but also on graph structure, spread-estimation accuracy, and the candidate-sharing budget.

References

- [1] D. Kempe, J. Kleinberg, and E. Tardos. Maximizing the spread of influence through a social network. *KDD*, 2003.
- [2] G. Nemhauser, L. Wolsey, and M. Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming*, 1978.
- [3] J. Leskovec et al. Cost-effective outbreak detection in networks. *KDD*, 2007.
- [4] B. Mirzasoleiman et al. Distributed submodular maximization: Identifying representative elements in massive data. *NeurIPS*, 2013.
- [5] Y. Tang, X. Xiao, and Y. Shi. Influence maximization: Near-optimal time complexity meets practical efficiency. *SIGMOD*, 2014.
- [6] Y. Tang, Y. Shi, and X. Xiao. Influence maximization in near-linear time: A martingale approach. *SIGMOD*, 2015.
- [7] L. Wang et al. Fast and space-efficient parallel algorithms for influence maximization. *PVLDB*, 2023.
- [8] Y. Chen, T. Dey, and A. Kuhnle. Scalable distributed algorithms for size-constrained submodular maximization in the MapReduce and adaptive complexity models. *JAIR*, 2024.
- [9] R. Barik et al. GreedyRIS: Scalable influence maximization using distributed streaming maximum cover. *JPDC*, 2025.